PolicyPilot demo script

Demo recording script (4 to 5 minutes)

This is the script for the recorded walkthrough. It is written to be spoken, so read it in your own words.

There are two ways to run PolicyPilot, and the video shows both:

	Where	Sign-in	Branch
Hosted demo	https://policypilot-om-thakur.onrender.com	Shared demo passcode (no accounts)	`main`
Local full stack	http://localhost:8080	Real sign-in through Keycloak (OpenID Connect): sign in, create account, sign out	`local-sign-in`

The application code is the same in both. Sign-in is switched on by configuration (`AUTH_MODE=oidc`), not by different code. Render's free plan runs a single web service, so the hosted demo leaves the identity server out and uses the passcode instead.

Before you press record

1. **Start the local stack with sign-in** (Docker Desktop must be running):

   ```
   git checkout local-sign-in
   scripts/run_with_sign_in.sh
   ```

   The first run creates `.env.auth-demo` with passwords for the test users `alice` and `bob`. Keep that file off screen.
2. Open http://localhost:8080 and wait until the **Knowledge base** shows **14 ready** documents.
3. **Do a dry run of every question below once.** The model's behaviour varies a little from run to run, and you want no surprises.
4. **Pre-upload nothing.** You will upload the assignment PDF live. Have it ready in a Finder window. Processing a PDF takes a little while, so you will start the upload, show other things, and come back.

5. Open the hosted URL in a second tab and wake it up (a free Render service can take about a minute).
6. Close notifications. Zoom the browser to about 125%. Hide `.env` files, the passcode, the OpenRouter and Render dashboards.

Script

0:00 to 0:20. What it is

Show the sign-in screen at http://localhost:8080.

> "This is PolicyPilot, an assistant that answers questions about university IT policies. It reads 14 PDFs, 10 university policies and 4 NIST documents, and every answer points to the exact page it came from. I'm running the full version locally, with real sign-in."

0:20 to 0:50. Sign in

Click **Sign in**, sign in as `alice` on the Keycloak page, and land back in the workspace.

> "Sign-in uses OpenID Connect with Keycloak. The browser never sees the identity token: the server exchanges it and gives the browser its own session cookie. Each user only sees their own conversations and uploads."

Point at the greeting and the account menu in the corner.

0:50 to 2:00. Ask questions and check an answer

Ask one of the suggested questions, for example:

> What bandwidth limits apply to unregistered guest devices?

While it runs, open the **Agent activity** panel.

> "On the right you can see what the agent is doing: its plan, the searches it runs and the pages it reads. It has a fixed budget: at most six tool calls per question."

When the answer appears, ask the cross-document question:

> A researcher has Level 3 confidential pre-publication data. What encryption is required at rest, and may the researcher paste it into a public consumer AI tool?

> "This one needs two different policies, the data classification standard and the AI guidance, and the answer cites both."

Manual check. Click a numbered citation chip under the answer.

> "Every claim carries a quote. Before the answer is shown, the code checks that this exact text is on the cited page. If it isn't, the claim is removed."

Click **View** to open the PDF at that page.

> "And here's the original page in the PDF, so anyone can check where the answer came from."

Close the PDF.

2:00 to 2:20. Upload a document

Open **Knowledge base**, click **Upload PDF** and choose the assignment PDF.

> "You can add your own documents. The upload is checked, split into pages and indexed in the background. Uploads are private to the user who added them."

Don't wait for it. It indexes while you show the design.

2:20 to 3:00. System design

Click **System design** in the sidebar. Move down the four rows, pointing at each one as you say it.

> "Here's how it's built. **Top row:** when a document comes in, it goes into a job queue, then gets parsed page by page, cut into small chunks, embedded and indexed in PostgreSQL. That's what's happening to my upload right now."

> "**Second row:** when a question comes in, the API checks who you are, then the agent loop plans, searches, reads pages and searches again if it needs to, at most six tool calls. The verifier checks every quote against the page before the answer is shown."

> "**Shared services:** PostgreSQL holds everything durable: documents, vectors, conversations and the job queue. Redis is only a cache; the app still works without it."

> "**Where it runs:** the same code runs on Render for reviewers, on Docker Compose locally with sign-in, and on Kubernetes, which I tested on a local cluster."

Go back to the workspace. If the upload shows as ready, ask one question about it, for example:

> What does the assignment require for the evaluation?

> "The answer cites the document I just uploaded, with its page number."

If it's still indexing, say so and move on. Don't wait on camera.

3:00 to 3:30. History, panels and clearing

Open **History**.

> "Conversations are kept here, so I can go back to an earlier question."

Click an earlier conversation, then toggle the side panel with **Collapse sidebar** and open it again.

> "The panels can be shown or hidden to give the answer more room."

Click the clear-history option and confirm.

> "And history can be cleared in one click, which starts a fresh session."

3:30 to 3:50. Sign out

Open the account menu and click **Sign out**.

> "Signing out ends the session on the server, not just in the browser, so the old cookie can't be reused."

You land back on the sign-in screen.

3:50 to 4:30. The hosted version

Switch to the Render tab.

> "The same application is hosted on Render for reviewers. Render's free plan runs one service, so the hosted version uses a shared demo passcode instead of the identity server, plus a rate limit and a daily spending cap. The code is the same; sign-in is a setting."

Enter the passcode (keep it off screen) and show that the 14 documents are there.

4:30 to 5:00. Close, with one honest limit

> "Two things I'd improve next. Deciding when to ask a clarifying question is the least reliable part; it works on the questions I tested but I wouldn't claim more than that. And the evaluation sets are small, so the numbers are indicative, not proof. The full results, the design and the limits are in the repository."

Stop recording.

If something goes wrong on camera

- **An answer is wrong or slow:** say so plainly, and move to the next question. It's more convincing than a perfect take.
- **The upload is still processing:** say "it's still indexing", and continue. Come back only if there's time.
- **Sign-in fails locally:** show the hosted version and talk through sign-in from `authentication.md`.

After recording

- Watch it once at normal speed. Check that no passcode, password, key or `.env` file appears in any frame.
- Upload it as **Unlisted**, open the link in a private window to confirm it plays without signing in, and add it to the README.

Saved evidence

Saved traces for the behaviours that are hard to trigger live are in `demo-evidence/`: a multi-document answer, a re-retrieval, a clarification with retained context, and a tool failure with one bounded retry (test environments only).